CS341 Software Engineering
Task Tracker: Project Report

Project 1 | CS341 Project | 1 November 2026

1. Project Overview

Task Tracker is a standalone desktop to-do list application implemented in Java 11 with Swing and SQLite accessed through JDBC. Different registered users can log in to the same local application, create tasks, assign tasks to other registered users and track completion. This simulates collaboration through sequential sessions on one computer; it does not provide real-time networking or cloud synchronization.

The assignment explicitly requires requirements analysis, design, implementation, unit testing, documentation and a presentation. The project also includes priority levels, optional deadlines and filtering/sorting as additional features.

2. Requirements and Scope

ID	Function	Acceptance criterion
FR-01	User registration	Create a unique username and a password-protected account.
FR-02	Authentication	Log in with valid credentials and clear the session on logout.
FR-03	Task creation	Store task title, creator, assignee and optional attributes.
FR-04	Task editing	The creator can edit title, description, assignee, priority and due date.
FR-05	Task deletion	The creator can permanently delete a task.
FR-06	Completion	The creator or assignee can complete and reopen a task.
FR-07	Assignment	Tasks can be assigned to registered users.
FR-08	Viewing	Users see only tasks they created or were assigned.
FR-09	Storage	SQLite tables are initialized on first run; data persists across launches.
FR-10	Additional: priority	Choose LOW, MEDIUM or HIGH.
FR-11	Additional: deadline	Optionally store a valid YYYY-MM-DD date.
FR-12	Additional: filtering	Filter open/completed/high-priority tasks and sort columns.

The application does not include notifications, a hosted server, multi-device synchronization, password recovery, or administrator privileges. These are intentionally outside the project brief.

3. Use Cases and Authorization

Visitor: register or log in. **Signed-in user:** create tasks, assign tasks and view/filter relevant tasks. **Creator:** edit, delete and toggle completion. **Assignee:** view and toggle completion. **Unrelated user:** cannot view or mutate another user's task.

The detailed creator/assignee permission policy is a design decision that makes the required behavior explicit; the brief does not mandate this exact permission matrix.

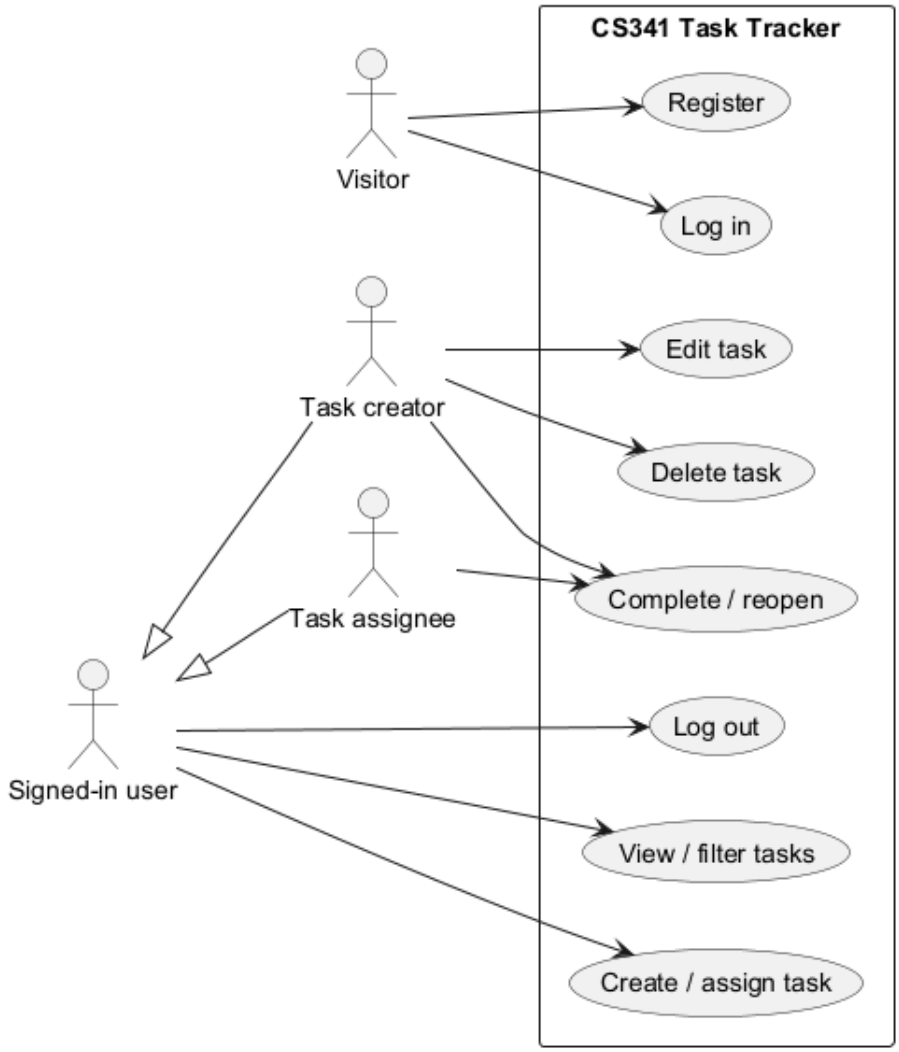


Figure 1: Use-case overview. The canonical editable source is `docs/plantuml/use_case.puml`.

4. System Architecture

The implementation is MVC-oriented with clear separation of responsibilities. `MainFrame` is the Swing view; `TaskTrackerController` manages the logged-in session and coordinates actions; `AuthService` and `TaskService` implement business rules; repositories encapsulate SQL; and `Database` creates connections and initializes the schema. Data objects are `User` and `TaskItem`.

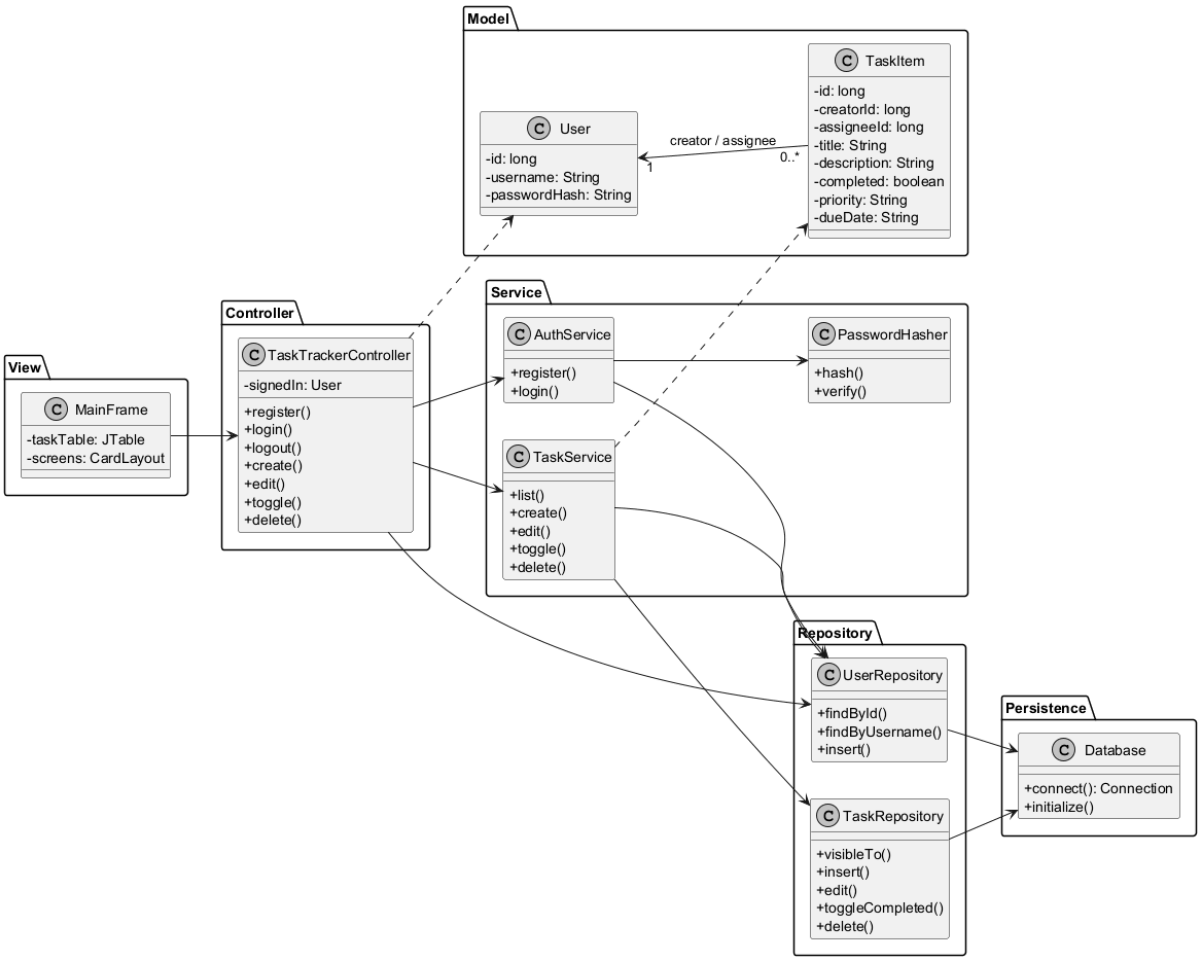


Figure 2: Class-level responsibilities and dependencies.

Layer	Responsibility
View	Swing forms, task table, actions and sorting/filtering.
Controller	Active user session and command dispatch.
Service	Input validation, role policy and authentication.
Repository	Parameterized SQL queries and conditional writes.
Database	JDBC connections, SQLite initialization and foreign-key enforcement.
Model	Immutable user and task data structures.

5. Database Design

The application creates one local `tasks.db` file. The `users` table contains a case-insensitive unique username and the password hash. The `tasks` table includes separate foreign keys to the creator and assignee. Indexes support filtering by creator, assignee and due date.

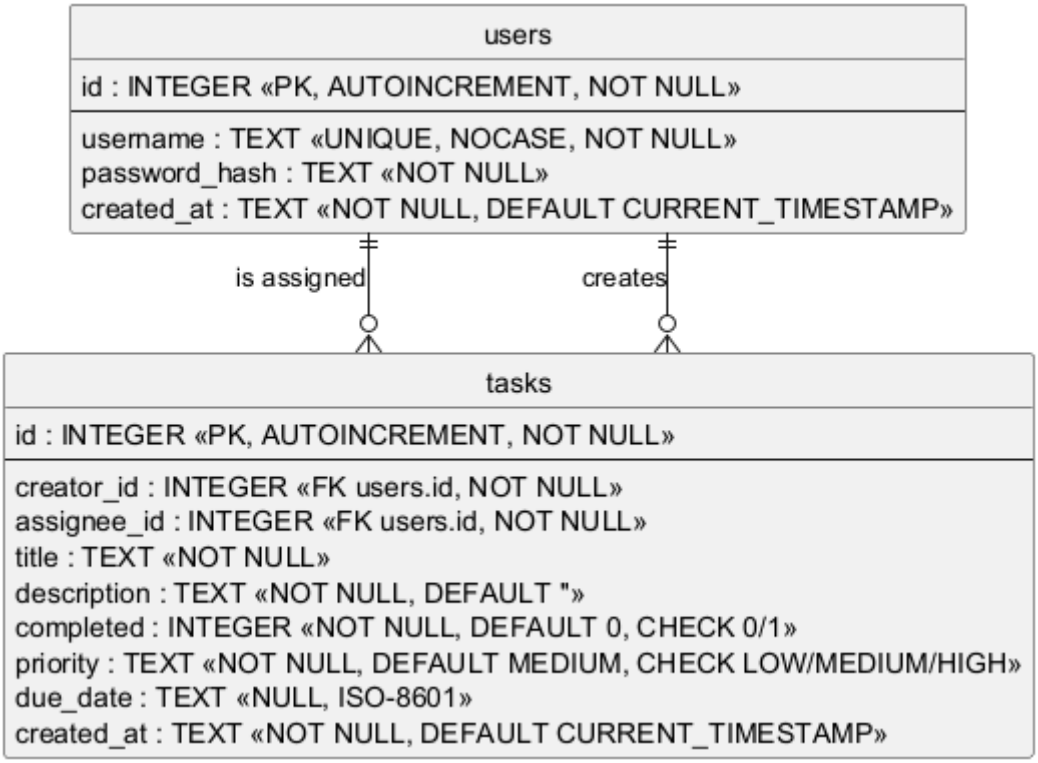


Figure 3: SQLite entity relationship diagram (two foreign keys into `users`).

Integrity: SQLite enforces a primary key, foreign keys, an enumeration constraint for priority and a Boolean-like completion constraint. Each JDBC connection executes `PRAGMA foreign_keys=ON`. SQL statements use bound parameters rather than assembling user-provided strings into SQL.

6. Implementation Decisions

- **Local-first application:** no database server or external runtime services are needed.

- **Password handling:** random per-user salts and PBKDF2-HMAC-SHA256 hashes; plaintext passwords are not stored in the database.
- **Authorization:** the service layer checks roles and SQL updates/deletions include the relevant user ID in their `WHERE` conditions.
- **Validation:** names, passwords, title lengths, priority values, dates and target users are checked before task writes.
- **User experience:** the same local database is used by different accounts; assignment, priority and filtering remain simple enough for a four-week coursework project.

7. Testing and Evidence

JUnit 5 integration and password tests are present in source code. They cover registration, credential verification, authorization, assignment, task lifecycle, validation, data persistence, SQLite constraints and injection-like strings. Four additional JUnit cases were added for direct repository authorization, self/other assignment, validation and reassignment.

Executed evidence: `mvn -B clean verify` passed all 17 JUnit 5 tests (0 failures, errors or skips) with SQLite JDBC; the packaged JAR completed a SQLite round trip; and 10 independent Python SQLite checks passed. The seven PlantUML diagrams were rendered to PNG and SVG. Manual end-to-end GUI acceptance, GitHub Actions and real team contributions remain unverified. The companion *Test Report* documents the evidence and exact acceptance scenarios.

8. Development Process and Deliverables

Week	Expected process / generated artifacts
1	Requirements, comparable application review, SQLite prototype and use cases.
2	MVC architecture, database schema, class diagrams and test plan.
3	Java implementation, organized packages, SQL schema and test source.
4	Executed tests, final report, revision notes and live presentation.

9. Limitations and Submission Requirements

- A graphical desktop is required for manual Swing acceptance testing.
- The local Maven build and real Surefire XML test results passed; see `docs/evidence/` and `target/surefire-reports/`.
- PlantUML `.puml` files are authoritative and all seven were rendered to SVG and PNG.
- The assignment requires a real team of 3–4 students, genuine member commits, a GitHub repository and a live demo. These cannot be fabricated by documentation.

10. Rubric Alignment

Assessment area	Weight
Requirements and design documentation	25%
Java implementation	30%
Testing and test report	20%
Code quality and OOP principles	15%
Presentation and demonstration	10%

11. References and Reproducibility

Primary brief: *CS341 Software Engineering, Project 1, Task Tracker – A Collaborative To-Do List Application (Java)*, supplied course PDF, five pages. For a limited competitor review, see `docs/COMPETITOR_REVIEW_EN.md`; this is qualitative context and not statistical survey evidence.

To reproduce: `mvn -B clean verify`, then `java -jar target/task-tracker-1.0.0.jar`. Source files, tests, SQL and exact reproduction instructions are in the project ZIP. Audit the outstanding verification gates in `docs/Gap_Audit_EN.pdf`.